context-mode — Full Technical Documentation

Version 1.0.0 | Elastic License 2.0 | April 2026

Table of Contents

1. Overview
2. Architecture
3. MCP Server & Tools
4. Knowledge Base
5. Sandbox Executor
6. Session Continuity
7. Hook System
8. Search Algorithm
9. Design Decisions
10. Configuration Reference
11. Platform Support

---

1. Overview

context-mode is a Cowork plugin for Claude Code that reduces context window consumption by up to 98%. It does this through three mechanisms:

- **Sandbox Execution**: Runs code in isolated subprocesses, returning only stdout. Raw file contents and command output never enter context.
- **Knowledge Base**: Indexes content into a local SQLite FTS5 database. Retrieves only relevant snippets via BM25 + trigram dual-strategy search.
- **Session Continuity**: Captures session events via hooks, builds priority-tiered snapshots before compaction, and restores session state afterward.

Problem Statement

Long Claude Code sessions in Cowork consume context rapidly. Every file read, web fetch, and shell command dumps raw output into the context window. A single Playwright snapshot can cost 56 KB. Twenty fetched documents can cost hundreds of KB. Over a long task, this bloats context, reduces response quality, and forces premature compaction — causing Claude to lose track of progress.

Solution

context-mode intercepts data-heavy operations and processes them outside the context window:

```
Without context-mode:           With context-mode:

┌──────────────────────┐        ┌──────────────────────┐
│ Context Window       │        │ Context Window       │
│                      │        │                      │
│ [60KB file content]  │        │ [120B search result] │
│ [45KB web page]      │        │ [80B cache hint]     │
│ [30KB cmd output]    │        │ [200B summary]       │
│                      │        │                      │
│ Total: 135KB         │        │ Total: 400B (99.7%)  │
└──────────────────────┘        └──────────────────────┘
```

2. Architecture

System Architecture

```
┌──────────────────────────────────────────────────────┐
│                    Cowork Session                    │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │
│  │ SessionStart │  │ PostToolUse  │  │ PreCompact   │ │
│  │    Hook      │  │    Hook      │  │    Hook      │ │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘ │
│         │                 │                 │        │
│         ▼                 ▼                 ▼        │
│  ┌──────────────────────────────────────────────────┐ │
│  │              MCP Server (stdio)                  │ │
│  │                                                  │ │
│  │  ┌──────────────┐  ┌──────────────────────────┐ │ │
│  │  │  Sandbox     │  │   Knowledge Base         │ │ │
│  │  │  Executor    │  │   (SQLite FTS5)          │ │ │
│  │  │              │  │                          │ │ │
│  │  │ ctx_execute  │  │ ctx_index                │ │ │
│  │  │ ctx_exec_file │  │ ctx_search               │ │ │
│  │  │ ctx_batch    │  │ ctx_fetch_and_index      │ │ │
│  │  └──────────────┘  └──────────────────────────┘ │ │
│  │                                                  │ │
│  │  ┌──────────────┐  ┌──────────────────────────┐ │ │
│  │  │  Session DB  │  │   Utilities              │ │ │
│  │  │  (SQLite)    │  │                          │ │ │
│  │  │              │  │ ctx_stats                │ │ │
│  │  │ Events       │  │ ctx_doctor               │ │ │
│  │  │ Snapshots    │  │ ctx_purge                │ │ │
│  │  └──────────────┘  └──────────────────────────┘ │ │
│  └──────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────┘
```

Data Flow

```
Tool Call (e.g., file read)
    │
    ▼
PostToolUse Hook ──► SessionDB (event stored)
    │
    ▼
MCP Tool (ctx_execute_file)
    │
    ├── File read in subprocess (raw content isolated)
    ├── Output indexed in FTS5 (if large)
    └── Compact result returned to context

    ...session continues...
```

```
PreCompact Hook fires
    │
    ├── Read all events from SessionDB
    ├── Build priority-tiered XML snapshot (≤2KB)
    └── Store in session_resume table

Context Compaction occurs

SessionStart Hook (source: "compact")
    │
    ├── Load snapshot from session_resume
    ├── Inject routing block + session guide
    └── Claude resumes with full awareness
```

Directory Structure

```
context-mode/
├── .claude-plugin/
│   └── plugin.json           # Cowork plugin manifest
├── .mcp.json                 # MCP server registration
├── .gitignore
├── package.json
├── server/
│   ├── index.js              # MCP server (9 tools, lifecycle)
│   ├── sandbox.js            # PolyglotExecutor
│   ├── knowledge.js          # ContentStore (FTS5)
│   ├── session.js            # SessionDB
│   ├── snapshot.js           # Compaction snapshot builder
│   ├── runtime.js            # Language runtime detection
│   ├── db-base.js            # SQLite utilities
│   ├── utils.js              # Query sanitization, Levenshtein
│   └── exit-classify.js      # Non-zero exit classification
├── hooks/
│   ├── hooks.json            # Hook definitions
│   ├── run-hook.cmd          # Cross-platform hook wrapper
│   ├── session-extract.js    # Event extraction (13 categories)
│   ├── session-helpers.js    # Session ID, paths, stdin
│   ├── routing-block.js      # Context routing instructions
│   ├── post-tool-use.js      # PostToolUse hook
│   ├── pre-compact.js        # PreCompact hook
│   ├── session-start.js      # SessionStart hook
│   └── subagent-stop.js      # SubagentStop hook
├── skills/                   # /ctx-stats, /ctx-doctor, /ctx-purge
├── agents/                   # context-optimizer agent
├── scripts/                  # setup.js, setup.sh
├── docs/                     # Landing page, full docs
└── test-e2e.js              # 137-test E2E suite
```

---

3. MCP Server & Tools

The MCP server (`server/index.js`) runs as a Node.js process communicating via stdio. It registers 9 tools:

Tool	Input	Output	Context Savings
`ctx_execute`	language, code, intent?	stdout only	94-100%
`ctx_execute_file`	files[], language, code	computed results	94-100%
`ctx_batch_execute`	commands[], queries[]	combined results + search	90-98%
`ctx_index`	content, source	chunk count confirmation	100% (content stored, not returned)
`ctx_search`	queries[], limit?	relevant snippets	N/A (retrieval)
`ctx_fetch_and_index`	url, queries?	cache hint or preview	95-99%
`ctx_stats`	(none)	session report	N/A
`ctx_doctor`	(none)	diagnostics	N/A
`ctx_purge`	confirm: true	confirmation	N/A

Auto-indexing Behavior

When `ctx_execute` output exceeds 5KB and an `intent` parameter is provided, the output is automatically indexed into the knowledge base and only the BM25-ranked snippets matching the intent are returned. When output exceeds 100KB without intent, it is indexed and a pointer is returned.

Progressive Search Throttling

`ctx_search` enforces a 60-second sliding window:

Calls in Window	Behavior
1-3	Full results (2 per query)
4-8	Reduced (1 per query) + warning
9+	Blocked, redirect to ctx_batch_execute

---

4. Knowledge Base

Database Schema

```sql
-- Content storage
CREATE TABLE sources (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  label TEXT NOT NULL,
  chunk_count INTEGER,
  code_chunk_count INTEGER,
  indexed_at TEXT DEFAULT (datetime('now'))
```

```sql
);

-- Porter stemmer FTS5
CREATE VIRTUAL TABLE chunks USING fts5(
  title, content, source_id UNINDEXED, content_type UNINDEXED,
  tokenize='porter unicode61'
);

-- Trigram FTS5
CREATE VIRTUAL TABLE chunks_trigram USING fts5(
  title, content, source_id UNINDEXED, content_type UNINDEXED,
  tokenize='trigram'
);

-- Vocabulary for fuzzy correction
CREATE TABLE vocabulary (word TEXT PRIMARY KEY);
```

Chunking Strategy

- **Markdown**: Split on `#` headings, keep code blocks intact, max 4096 bytes per chunk, paragraph split fallback
- **JSON**: Recursive walk with key paths as titles, array items batched by size
- **Plain text**: Blank-line sections first, fallback to fixed-size line groups with 2-line overlap

TTL Cache

`ctx_fetch_and_index` checks `sources.indexed_at` before fetching. If age < 24 hours, returns a cache hint (~40 bytes) instead of re-fetching. Estimated bytes saved: `chunkCount * 1600`.

---

5. Sandbox Executor

Process Isolation

- Each execution creates a unique temp directory: `mkdtempSync(join(OS_TMPDIR, '.ctx-mode-'))`
- Shell runs in project root (for git/relative paths); other languages run in temp dir
- Environment stripped of `NODE_OPTIONS` and `ELECTRON_RUN_AS_NODE`
- Hard stdout cap: 100MB (prevents `yes` or `/dev/urandom` from consuming memory)
- Default timeout: 30 seconds
- Background mode: detaches process after timeout, returns partial output

Platform-Specific Process Management

Platform	Process Kill Method
Windows	`taskkill /F /T /PID <pid>` (tree kill)
macOS/Linux	`process.kill(-pid, 'SIGKILL')` (process group)

Language-Specific Wrapping

Language	Wrapping
Go	Adds `package main` if missing

PHP	Adds `<?php` opening tag if missing
Shell	Adds `#!/usr/bin/env bash` + `set -e` if no shebang
Rust	Compile-then-run (rustc to binary, then execute)

---

6. Session Continuity

Event Categories (13 types, 4 priority levels)

Priority	Category	Types
1 (Critical)	file	file_read, file_write, file_edit
1 (Critical)	task	task, task_update
1 (Critical)	rule	rule, rule_content
2 (High)	error	error_tool
2 (High)	cwd	cwd
2 (High)	env	env (package installs)
3 (Normal)	git	git (commit, push, merge, etc.)
3 (Normal)	subagent	subagent, subagent_complete
3 (Normal)	skill	skill
3 (Normal)	mcp	mcp_call
4 (Low)	data	data

Deduplication

SHA-256 hash of `type + category + data`, checked against last 5 events. Duplicates rejected.

FIFO Eviction

Max 1000 events per session. When exceeded, lowest-priority event is evicted first.

Compaction Snapshot

Budget: 2048 bytes. Sections included (priority order):

1. Files (P1) — last 10 active files with operation counts
2. Rules (P1) — CLAUDE.md and project rules
3. Task state (P1) — pending/in-progress tasks
4. Errors (P2) — last 5 errors
5. Git (P3) — recent git operations
6. Decisions (P3) — key decisions made
7. Subagents (P3) — sub-agent tasks
8. Environment (P4) — last cwd, env changes

Lower-priority sections are dropped first if budget is tight. Each section includes `ctx_search` hints for retrieving full details.

---

7. Hook System

Hook Events Used

Event	Script	Purpose
SessionStart	session-start.js	Inject routing block, handle compact resume
PostToolUse	post-tool-use.js	Capture session events (<20ms)
PreCompact	pre-compact.js	Build and store resume snapshot
SubagentStop	subagent-stop.js	Capture sub-agent outcomes

Cross-Platform Hook Execution

All hooks use `run-hook.cmd` — a polyglot CMD/bash wrapper:

- **Windows**: CMD portion calls `node hooks/<script>.js`
- **macOS/Linux**: Bash portion calls `node hooks/<script>.js`

---

8. Search Algorithm

Three-Layer Pipeline

```
Query: "BM25 ranking algorithm"
    │
    ▼
Layer 1: Porter Stemmer FTS5
    bm25(chunks, 5.0, 1.0)  ← title weighted 5x
    Results: [{title, content, rank, highlighted}, ...]
    │
    ▼
Layer 2: Trigram FTS5
    bm25(chunks_trigram, 5.0, 1.0)
    Results: [{title, content, rank, highlighted}, ...]
    │
    ▼
RRF Fusion (K=60)
    score = Σ 1/(60 + rank + 1) across both layers
    Merge by source_id::title key
    │
    ▼
Proximity Reranking (multi-term queries)
    findMinSpan(position_lists)
    boost = 1 / (1 + minSpan / contentLength)
    │
    ▼
If empty → Levenshtein Fuzzy Correction
```

```
    Edit distance thresholds: 1-4 chars → 1, 5-12 → 2, 13+ → 3
    Re-run RRF with corrected terms
    │
    ▼
Smart Snippets
    300-char windows around matches, merge overlapping
    Up to 1500 chars total
```

---

9. Design Decisions

Why SQLite FTS5 over vector search?

FTS5 with BM25 provides deterministic, fast, and dependency-free full-text search. No embedding model needed, no API calls, no network dependency. The dual-tokenizer approach (Porter + trigram) with RRF fusion achieves search quality comparable to semantic search for code and technical documentation while running entirely locally.

Why subprocess isolation instead of in-process eval?

Security and resource control. Subprocess isolation prevents:

- Runaway code from blocking the MCP server
- Memory leaks from accumulating in the server process
- File system or network access from escaping the sandbox
- stdout/stderr from polluting the MCP transport

Why ≤2KB snapshot budget?

The snapshot is injected into context after compaction. Every byte of the snapshot is a byte that can't be used for the user's actual work. 2KB is enough to capture active files, pending tasks, recent errors, and search hints — the minimum Claude needs to resume effectively.

Why progressive search throttling?

Without throttling, Claude can fall into a search loop: search → not quite right → search again → refine → search again. Each search adds results to context. The throttle curve (2→1→blocked) forces batching, which is more context-efficient.

---

10. Configuration Reference

Key Constants

Constant	Value	Purpose
MAX_CHUNK_BYTES	4096	Max bytes per FTS5 chunk
BM25 weights	bm25(chunks, 5.0, 1.0)	Title weighted 5x
RRF_K	60	Reciprocal Rank Fusion constant
TTL_MS	86,400,000 (24h)	Fetch cache TTL
SEARCH_WINDOW_MS	60,000 (60s)	Throttle window
SEARCH_BLOCK_AFTER	8	Block threshold

INTENT_SEARCH_THRESHOLD	5,000	Auto-index stdout threshold
HARD_CAP_BYTES	104,857,600 (100MB)	Executor stdout limit
MAX_EVENTS_PER_SESSION	1,000	FIFO eviction cap
DEDUP_WINDOW	5	Recent events checked for duplicates
MAX_SNAPSHOT_BYTES	2,048	Compaction snapshot budget

Environment Variables

Variable	Source	Purpose
`CLAUDE_PLUGIN_ROOT`	Cowork	Absolute path to plugin installation directory
`CLAUDE_PLUGIN_DATA`	Cowork	Persistent data directory (survives updates)
`CLAUDE_PROJECT_DIR`	Cowork	Current project working directory
`CLAUDE_SESSION_ID`	Cowork	Current session identifier
`NODE_PATH`	.mcp.json	Points to `${CLAUDE_PLUGIN_DATA}/node_modules`

---

11. Platform Support

Windows

- Shell execution via Git Bash (`C:\Program Files\Git\usr\bin\bash.exe`)
- Process tree kill via `taskkill /F /T /PID`
- Temp directory: `%TEMP%` or `%TMP%`
- Runtime detection: `where` command (stderr suppressed)
- Hook execution: `run-hook.cmd` (CMD polyglot)

macOS

- Shell execution via `/bin/bash` or `/bin/zsh`
- Process group kill via `kill(-pid, SIGKILL)`
- Temp directory: `getconf DARWIN_USER_TEMP_DIR`
- Runtime detection: `command -v`
- Hook execution: `run-hook.cmd` (bash polyglot)

Cross-Platform Guarantees

- All file paths use `path.join()` , never hardcoded separators
- Dynamic imports use `pathToFileURL()` for ESM compatibility on Windows
- better-sqlite3 ships pre-built binaries for both platforms via npm
- No platform-specific dependencies